

Trading the Trend

"The market is going up," said Tom trendedly.

Having an Instantaneous Trendline with zero lag (Equations 2.8 and 2.9) is a good beginning to generate a responsive trend-following system. The system would be even more responsive if it contained a trigger that preceded the Instantaneous Trendline rather than following it and offering a confirming signal. A leading trigger can be generated by adding a two-day momentum of the Instantaneous Trendline to the Instantaneous Trendline itself.

The rationale for the leading trigger is that adding the two-day momentum to the current value in a trend is predicting where the Instantaneous Trendline will be two days from now. When plotting the trigger on the current bar, the trigger must lead the Instantaneous Trendline by two bars. On a more mathematical level, the lag of the trigger is shown in Figure 3.1. The figure shows that the low-frequency lead is two bars and the worst-case lag occurs at a frequency of 0.25 cycles per day (a four-bar cycle period). The lag is of no concern because the attenuation of the Instantaneous Trendline (shown in Figure 2.5) makes the amplitude of the components in the vicinity of 0.25 cycles per day almost irrelevant to the overall response.

There is a price to pay for achieving the lead response of the trigger. That price is that leading functions cause a higher-frequency gain in the filter instead of attenuation, which has a smoothing effect. Therefore, high-frequency gain causes the resulting transfer response to look more ragged than the original function. This is the case for any momentum function. The gain response of the trigger has a maximum of 9.5 dB at a frequency of 0.25 cycles per day, as shown in Figure 3.2. In this case, the gain does not

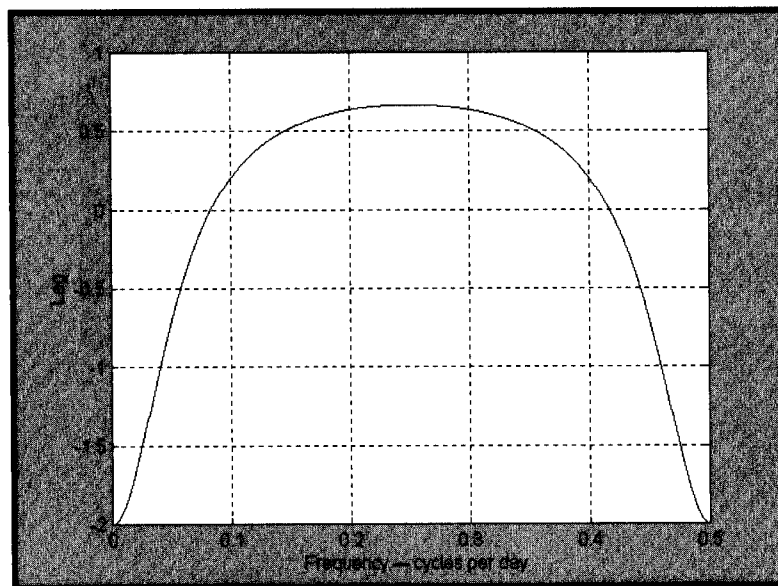


FIGURE 3.1 Lead and Lag of the Trigger as a Function of Frequency

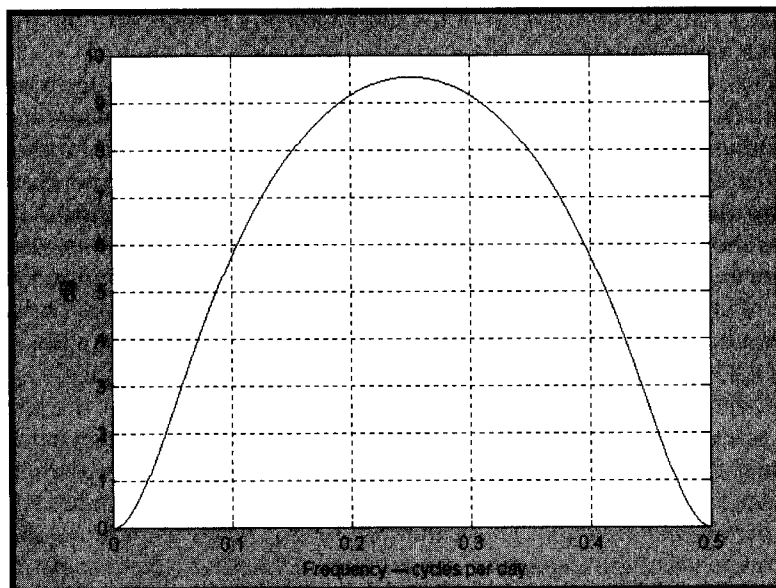


FIGURE 3.2 Gain Response of the Trigger

severely affect the smoothness of the trigger because the Instantaneous Trendline has an attenuation of 26 dB at 0.25 cycles per day, as shown in Figure 2.5. Therefore, using both terms to compute the net attenuation, the worst-case high-frequency smoothing attenuation is still about 16 dB. This means the trigger will have about the same degree of smoothness as the Instantaneous Trendline.

The Instantaneous Trendline and the Trigger of the trend-following system are shown as indicators in Figure 3.3; the EasyLanguage code to create these indicator lines is shown in Figure 3.4, and the eSignal Formula Script (EFS) code is shown in Figure 3.5. The process for creating a trend-following trading system from the indicators is simple. One unique aspect of the code is that the ITrend is forced to be a finite impulse response (FIR)-smoothed version of price for the first seven bars of the calculation. This initialization is included to cause the ITrend to converge more rapidly to its correct value from the beginning transient. The strategy enters a long position when the trigger crosses over the Instantaneous Trendline and enters a short position when the trigger crosses under the Instantaneous Trendline. However, an effective trading system is more than following a simple set of indicators.

First, experience has shown that greater profits result from using limit orders rather than market orders or stop orders. Market orders are self-explanatory. Stop orders mean the market must be going in the direction of the trade before the order is filled. For example, for long-position trades, the stop order must be placed above the current price. Thus, the price must

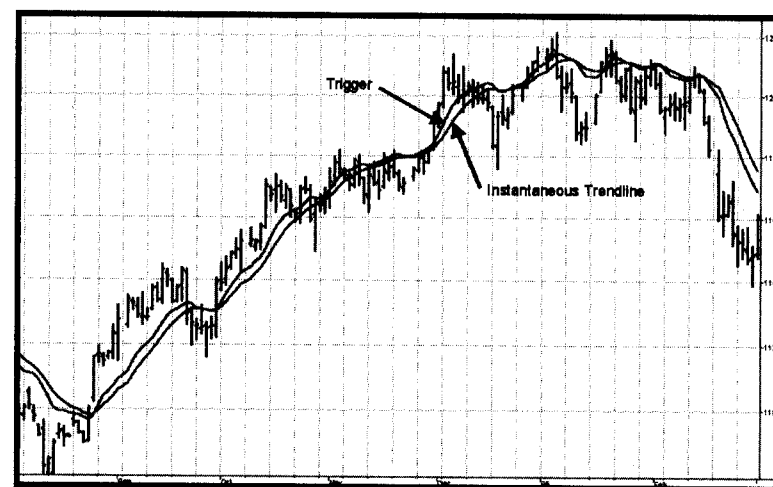


FIGURE 3.3 Crossing of the Trigger and Instantaneous Trendline are Trading Signals

```

Inputs:      Price((H+L)/2),
             alpha(.07);

Vars:        Smooth(0),
             ITrend(0),
             Trigger(0);

ITrend = (alpha - alpha*alpha/4)*Price
        + .5*alpha*alpha*Price[1] - (alpha
        - .75*alpha*alpha)*Price[2] + 2
        *(1 - alpha)*ITrend[1] - (1 - alpha)
        *(1 - alpha)*ITrend[2];
If currentbar < 7 then ITrend = (Price + 2*Price[1]
        + Price[2]) / 4;
Trigger = 2*ITrend - ITrend[2];

Plot1(ITrend, "ITrend");
Plot2(Trigger, "Trig");

```

FIGURE 3.4 EasyLanguage Code for the ITrend Indicator

increase from its current level before you get stopped into the long-position trade. This means you necessarily give up some of the profits you would otherwise have gotten if you had entered on a market order at the instant of your signal. You can lose additional profits from stop orders due to slippage. Slippage is the difference between your stop value and the price at which your order actually got filled. In fast markets slippage can be substantial. If limit orders are placed for the long position, the limit price must be below the current price. That is, the market must move against your anticipated trade before you get a fill. This means that if the price drops sufficiently so that your limit order is filled, you have captured additional profits if the price subsequently reverses and goes in the direction of your signal. Furthermore, if there is any slippage in filling the limit order, the slippage will be negative because it is going in the direction opposite to your intended trade. When the price turns around and goes in the direction of your signals, you have therefore captured the slippage as profit. In the EasyLanguage trading strategy code of Figure 3.6, I have set the level of the limit order to be 35 percent of the current bar's range added onto the closing price of the current bar (in the case of a short signal) or subtracted from the closing price of the current bar (in the case of a long signal). The 35 percent is the input variable `RngFrac`, and is an optimizable parameter.

```

/*****
Title:      Instantaneous Trendline
*****/

function preMain() {
    setPriceStudy(true);
    setStudyTitle("Instantaneous Trendline");
    setCursorLabelName("IT", 0);
    setDefaultBarThickness(2, 0);
}

var a = 0.05;
var IT = 0;
var IT1 = 0;
var IT2 = 0;
var Price = 0;
var Price1 = 0;
var Price2 = 0;

function main() {
    if (getBarState() == BARSTATE_NEWBAR) {
        IT2 = IT1;
        IT1 = IT;
        Price2 = Price1;
        Price1 = Price;
    }

    Price = close();

    IT = (a-((a/2)*(a/2)))*Price + ((a*a)/2)*Price1
        - (a-(3*(a*a))/4)*Price2 + 2*(1-a)*IT1
        - ((1-a)*(1-a))*IT2;

    return (IT);
}

```

FIGURE 3.5 EFS Code for the ITrend Indicator

Unfortunately, not all trading signals are perfect. In fact, with the crossover strategy that I have developed it is possible to be on the wrong side of the trade for a substantial period from time to time. For this reason, I have added a rule that if the price goes against your position by more than some percentage, the strategy will correct itself and automatically reverse to the opposite position. The percentage is supplied as the input variable

```

Inputs:      Price((H+L)/2),
             alpha(.07),
             RngFrac(.35),
             RevPct(1.015);

Vars:        Smooth(0),
             ITrend(0),
             Trigger(0);

ITrend = (alpha - alpha*alpha/4)*Price
         + .5*alpha*alpha*Price[1] - (alpha
         - .75*alpha*alpha)*Price[2] + 2
         *(1 - alpha)*ITrend[1] - (1 - alpha)
         *(1 - alpha)*ITrend[2];
If currentbar < 7 then ITrend = (Price + 2*Price[1]
+ Price[2]) / 4;
Trigger = 2*ITrend - ITrend[2];

If Trigger Crosses Over ITrend then Buy Next Bar at
Close - RngFrac*(High - Low) Limit;
If Trigger Crosses Under ITrend then Sell Short Next
Bar at Close + RngFrac*(High - Low) Limit;

If MarketPosition = 1 and Close < EntryPrice/RevPct
then Sell Short Next Bar On Open;
If MarketPosition = -1 and Close > RevPct*EntryPrice
then Buy Next Bar on Open;

```

FIGURE 3.6 EasyLanguage Code for the Instantaneous Trendline Trading Strategy

RevPct. RevPct is an optimizable parameter, but I find that the default value of 1.5 percent (RevPct = 1.015) is a relatively robust number. The same strategy for EFS code is given in Figure 3.7.

I applied the strategy code of Figures 3.6 and 3.7 to several currency futures because it is well known that currencies tend to trend. I additionally introduced a \$2,500 money management stop to further avoid giving back accumulated profits. Doing this, I achieved the trading results shown in Table 3.1. The time span is on the order of a quarter century, and a relatively large number of trades are taken. The Instantaneous Trend Strategy consists of only a few independent parameters. Since the ratio of the number of trades to the number of parameters is large and since the trading took place over a large time span, it is highly unlikely that the strategy has

```

/*****
Title:      ITrend Trading Strategy
Coded By:   Chris D. Kryza (Divergence Software, Inc.)
Email:      c.kryza@gte.net
Incept:     06/27/2003
Version:    1.0.0

=====
Fix History:

06/27/2003 - Initial Release
1.0.0

=====
*****/

//External Variables

var grID           = 0;
var nBarCount      = 0;
var xOver          = 0;
var nStatus        = 0;
var nEntryPrice    = 0;
var nDirection     = 0;
var nLimitPrice    = 0;
var nAdj1          = null;

var aPriceArray    = new Array();
var aITrendArray   = new Array();

//== PreMain function required by eSignal to set_
   things up
function preMain() {
var x;

    setPriceStudy(true);
    setStudyTitle("ITrend Trading Strategy");
    setCursorLabelName("ITrend", 0);
    setCursorLabelName("Trig", 1);

```

(continued)

FIGURE 3.7 EFS Code for the Instantaneous Trendline Trading Strategy

```

setDefaultBarFgColor( Color.blue, 0 );
setDefaultBarFgColor( Color.red, 1 );

//initialize arrays
for (x=0; x<10; x++) {
    aPriceArray[x]          = 0.0;
    aITrendArray[x]         = 0.0;
}

}

//== Main processing function
function main( Alpha, RngFrac, RevPct ) {
    var x;
    var nPrice;

    if (getCurrentBarIndex() == 0) return;

    //initialize parameters if necessary
    if ( Alpha == null ) {
        Alpha = 0.07;
    }
    if ( RngFrac == null ) {
        RngFrac = 0.35;
    }
    if ( RevPct == null ) {
        RevPct = 1.015;
    }

    // study is initializing
    if (getBarState() == BARSTATE_ALLBARS) {
        return null;
    }

    if (nAdj1 == null) nAdj1 = (high()-low()) * 0.20;

    //on each new bar, save array values
    if ( getBarState() == BARSTATE_NEWBAR ) {

        nBarCount++;

        aPriceArray.pop();
        aPriceArray.unshift( 0 );

```

FIGURE 3.7 (Continued)

```

        aITrendArray.pop();
        aITrendArray.unshift( 0 );

    }

    nPrice = ( high()+low() ) / 2;
    aPriceArray[0] = nPrice;

    if (aPriceArray[2] == 0) return;

    if ( nBarCount < 7 ) {
        aITrendArray[0] = (nPrice
            + 2*aPriceArray[1]
            + aPriceArray[2])/4;
    }
    else {
        aITrendArray[0] = (Alpha
            - Alpha*Alpha/4)*nPrice
            + 0.5*Alpha*Alpha*aPriceArray[1]
            - (Alpha - 0.75*Alpha*Alpha)
            * aPriceArray[2] + 2*(1-Alpha)
            *aITrendArray[1] - (1-Alpha)
            *(1-Alpha)*aITrendArray[2];
    }

    if (aITrendArray[2] == 0) return;

    nTrig = 2 * aITrendArray[0] - aITrendArray[2];

    nStatus = 0;
    if ( Strategy.isLong() ) nStatus = 1;
    if ( Strategy.isShort() ) nStatus = -1;

    var bReverseTrade = false;
    if ( nStatus == 1 && close()
        < (nEntryPrice/RevPct) ) {
        ReverseToShort();
        bReverseTrade = true;
    } else if ( nStatus == -1 && close()
        > (RevPct*nEntryPrice) ) {
        ReverseToLong();
        bReverseTrade = true;
    }
}

```

(continued)

FIGURE 3.7 (Continued)

```

    }
    //check for new signals
    if (bReverseTrade == false) {
        if ( nTrig > aITrendArray[0] ) {
            if ( xOver == -1 && nStatus != 1) {
                nLimitPrice = Math.max(low(), (close()
                    - ( high()-low() ) *RngFrac));
                LongLimit( nLimitPrice );
                nDirection = 1;
            }
            xOver = 1;
        } else if ( nTrig < aITrendArray[0] ) {
            if ( xOver == 1 && nStatus != -1) {
                nLimitPrice = Math.min(high(), (close()
                    + ( high()-low() ) *RngFrac));
                ShortLimit( nLimitPrice );
                nDirection = -1;
            }
            xOver = -1;
        }
    }

    if (!isNaN( aITrendArray[0] ) ) {
        return new Array( aITrendArray[0],_
            nTrig );
    }
}

function LongLimit( nPrice ) {
    Strategy.doLong("Long", Strategy.LIMIT,_
        Strategy.THISBAR, Strategy.DEFAULT,_
        nPrice );
    nEntryPrice = nPrice;
    drawShapeRelative(0, low()-nAdj1, Shape.UPARROW,_
        "", Color.lime, Shape.ONTOP, gID());
    return;
}

function ShortLimit( nPrice ) {
    Strategy.doShort("Short", Strategy.LIMIT,_
        Strategy.THISBAR, Strategy.DEFAULT,_
        nPrice );

```

FIGURE 3.7 (Continued)

```

    nEntryPrice = nPrice;
    debugPrintln(getCurrentBarIndex()
        + " short " + nPrice);
    drawShapeRelative(0, high()+nAdj1, Shape.DOWNARROW,_
        "", Color.maroon, Shape.ONTOP, gID());
    return;
}

function ReverseToLong() {
    Strategy.doLong("Reverse to Long",_
        Strategy.MARKET, Strategy.NEXTBAR,_
        Strategy.DEFAULT );
    DrawShapeRelative(1, low(1)-nAdj1,_
        Shape.UPARROW, "", Color.lime,_
        Shape.ONTOP, gID());
    nEntryPrice = open(1);
    nStatus = 1;
    nDirection = 0;
    nLimitPrice = 0;
    return;
}

function ReverseToShort() {
    Strategy.doShort("Reverse to Short",_
        Strategy.MARKET, Strategy.NEXTBAR,_
        Strategy.DEFAULT );
    drawShapeRelative(1, high(1)+nAdj1,_
        Shape.DOWNARROW, "", Color.maroon,_
        Shape.ONTOP, gID());
    nEntryPrice = open(1);
    nStatus = -1;
    nDirection = 0;
    nLimitPrice = 0;
    return;
}

//== gID function assigns unique identifier to_
//graphic/text routines
function gID() {
    grID ++;
    return( grID );
}

```

FIGURE 3.7 (Continued)

TABLE 3.1 Sample Trading Results Using the Instantaneous Trendline Strategy

Future	Net Profit	Number of Trades	Percent Profitable	Profit Factor	Max DD
EC (4/81–3/03)	\$201,812	230	42.2%	1.89	(\$26,775)
JY (9/81–3/03)	\$221,312	229	48.5%	2.50	(\$11,712)
SF (6/76–3/03)	\$129,175	337	45.1%	1.52	(\$15,387)

been curve fitted. Curve fitting is a weakness of many technical analysis trading strategies.

Please allow me to brag about the Instantaneous Trendline Strategy. (Perhaps it is not bragging, because as Muhammed Ali said, “It ain’t bragging if you can really do it.”) The performance results of this strategy are comparable to, or exceed, the performance of commercial systems costing thousands of dollars. You can create synthetic equity growth curves using the established percentage of profitable trades and profit factors. This is explained in Chapter 15. You will find the equity growth trading the currencies in Table 3.1 to be remarkably consistent.

KEY POINTS TO REMEMBER

- The Instantaneous Trendline has zero lag.
- The Instantaneous Trendline has about the same smoothing as an exponential moving average (EMA) using the same alpha.
- The smoothing enables the use of a trading trigger that has a two-bar lead.
- Trading signals are generated by the crossing of the Trigger line and the Instantaneous Trendline.
- Trade entries are made on limit orders to capture a larger range of the trade and to eliminate slippage losses.
- Major losses are avoided by recognizing when a trade is on the wrong side and reversing position.
- The Instantaneous Trendline Strategy can be optimized for application to many stocks and commodity markets.

CHAPTER 4

Trading the Cycle

“It happens again and again,” said Tom periodically.

Equation 2.5 described a high-pass filter that isolated the cycle mode components. Essentially all that need be done to generate a cycle-based indicator is to plot the results of this equation. However, some smoothing is required to remove the two-bar and three-bar components that detract from the interpretation of the cyclic signals. These components can be removed with a simple finite impulse response (FIR)¹ low-pass filter as

$$\text{Smooth} = (\text{Price} + 2 * \text{Price}[1] + 2 * \text{Price}[2] + \text{Price}[3]) / 6; \quad (4.1)$$

The lag of the Smooth filter of Equation 4.1 is 1.5 bars at all frequencies. Figure 4.1 demonstrates that the Smooth filter eliminates the two- and three-bar cycle components. The Smooth filter is to be used as an additional filter to remove the distracting very-high-frequency components, thus creating an indicator that is easier to interpret for trading.

The EasyLanguage code to make a cycle component indicator is given in Figure 4.2 and the eSignal Formula Script (EFS) code is given in Figure 4.3. I call this the Cyber Cycle Indicator. After the inputs and variables are defined, the smoothing filter of Equation 4.1 and the high-pass filter of Equation 2.7 are computed. They are followed by an initialization condition that facilitates a rapid convergence at the beginning of the input data. A trading trigger signal is created by delaying the cycle by one bar.

Trading the Cyber Cycle Indicator is straightforward. Buy when the Cycle line crosses over the Trigger line. You are at the bottom of the cycle